Задача N1

Пусть G — связный граф с минимум тремя вершинами. Все циклы и пути подразумеваются рёберно-простыми. Рассмотрим следующие условия:

(a) В графе G нет мостов.

(b) Для любых двух вершин u и v существуют два рёберно-простых пути между u и v, не имеющие общих ребер.

(c) Любые две вершины графа принадлежат какому-либо циклу.

(d) Для любой вершины u и любого ребра e существует цикл, содержащий и u, и e.

(e) Для любых двух рёбер e_1 и e_2 существует цикл, содержащий и e_1, и e_2.

(f) Для любых двух вершин u, v и любого ребра e найдётся путь из u в v, проходящий через e.

(g) Для любых двух вершин u, v и любого ребра e найдётся путь из u в v, не проходящий через e.

(h) Для любых трёх вершин u, v, w найдётся путь из u в v, проходящий через w.

Мы покажем, что все эти условия эквивалентны. Основная идея состоит в том, что каждое из них формулирует, по-своему, свойство 2-реберносвязности (без мостов) графа.

(a) $\iff$ (b):
Если в G нет мостов, то удаление любого ребра не разрывает связность, и, согласно версии теоремы Мензера для рёбер, между любыми двумя вершинами существует два рёберно-простых пути, не имеющие общих ребер. Обратно, если для любых двух вершин существуют два такие пути, то никакое ребро не может быть единственным связующим звеном (то есть мостом).

(b) $\implies$ (c):
Пусть для любых вершин u и v существуют два рёберно-простых пути, не имеющих общих ребер. Тогда объединение этих двух путей содержит цикл, поскольку если пути различны, их объединение содержит хотя бы одну замкнутую последовательность рёбер, проходящую через u и v. Таким образом, u и v принадлежат некоторому циклу.

(c) $\implies$ (d):
Пусть любые две вершины лежат в некотором цикле. Рассмотрим произвольную вершину u и ребро e, соединяющее, скажем, вершины x и y. Так как граф связен, существует путь от u до, например, x. По условию (c) вершины u и y лежат в некотором цикле C. Если ребро e не содержится в C, то,

используя связность, можно «подтянуть» e к циклу (например, найти путь от x до y, не содержащий e, а затем добавить e). В любом случае можно построить цикл, содержащий как u, так и e.

(d) $\Longrightarrow$ (e):

Пусть для любой вершины u и любого ребра e существует цикл, содержащий их. Возьмём произвольные ребра e_1 и e_2. Выберем конечную вершину u, инцидентную e_1. Тогда по условию (d) существует цикл, содержащий u и e_2. Но поскольку u принадлежит e_1, этот цикл содержит и e_1, и e_2.

(e) $\Longrightarrow$ (a):

Если любые два ребра лежат в некотором цикле, то каждое ребро содержится в некотором цикле. Но ребро, лежащее в цикле, не может быть мостом, поскольку удаление его сохраняет связность цикла. Следовательно, в графе нет мостов.

(a) $\Longrightarrow$ (g):

Если в G нет мостов, то удаление любого ребра не разрывает связность графа. Это означает, что для любых двух вершин u и v найдётся путь, не использующий данное ребро e.

(a) $\Longrightarrow$ (f):

Пусть G не содержит мостов, тогда каждое ребро принадлежит некоторому циклу (см. (e)). Рассмотрим произвольные вершины u, v и ребро e. Так как граф связен и e лежит в некотором цикле C, можно найти путь от u до одного из вершин C, затем пройти по циклу через ребро e, и, наконец, от другого конца цикла дойти до v. Таким образом, существует путь из u в v, проходящий через e.

(f) $\Longleftrightarrow$ (g):

Если для любых вершин u, v и ребра e найдётся путь, проходящий через e, то, в частности, граф обладает избыточностью рёбер, что гарантирует существование альтернативного пути, не использующего e. Обратно, если для любых вершин существует путь, избегающий произвольного ребра, то в силу цикличности можно «вынудить» прохождение через e, используя дополнительный обход по циклу. Таким образом, оба условия выражают свойство 2-реберносвязности графа.

(a) $\Longrightarrow$ (h):

Пусть граф не содержит мостов. Рассмотрим любые три вершины u, v, w. Если w лежит на каком-либо фиксированном пути между u и v, требуемое выполнено. Если же нет, то, поскольку по (a) граф 2-реберносвязный, по условию (f) или (g) можно найти альтернативный путь от u до v, проходящий через w. Таким образом, для любых трёх вершин существует путь от u до v, содержащий w.

(h) $\Longrightarrow$ (c):

Если для любых трех вершин существует путь от u до v, проходящий через произвольную вершину w, то, взяв для w некоторую вершину, отличную

от u и v, можно построить цикл, содержащий u и v. Действительно, пусть путь от u до v, проходящий через w, обозначен как $u \to \cdots \to w \to \cdots \to v$. Добавив ребро uv (такое ребро может не существовать, но поскольку граф связен, существует альтернативный путь, замыкающий этот маршрут в цикл), получим цикл, в котором содержатся u и v. (Формально, условие (h) свидетельствует о сильной связности графа; в конечном итоге, оно приводит к тому же циклическому свойству, что и условие (c).)

Таким образом, мы показали цепочку импликаций, приводящую к равносильности всех приведённых условий.

Вывод. Для связного графа G с хотя бы тремя вершинами следующие условия эквивалентны:

$$\text{(a)} \iff \text{(b)} \iff \text{(c)} \iff \text{(d)} \iff \text{(e)} \iff \text{(f)} \iff \text{(g)} \iff \text{(h)}.$$

Задача N2

Пусть G — связный граф, содержащий хотя бы три вершины, и все циклы и пути подразумеваются *вершинно-простыми* (то есть без повторения внутренних вершин). Рассмотрим следующие условия:

(a) В графе G нет точек сочленения (cut-vertices).

(b) Для любых двух вершин u и v существуют два пути, соединяющих u и v, не имеющих общих промежуточных вершин.

(c) Любые две вершины графа G принадлежат некоторому циклу.

(h) Для любых трёх вершин u, v, w найдётся путь из u в v, не проходящий через w.

Ниже приведём доказательство эквивалентности этих четырёх условий.

(a) $\iff$ (b).
Известно (теорема Менгера для вершин), что граф не имеет точек сочленения, то есть является 2-связным, если и только если для любых двух вершин существует два *внутренне-вершинно-дисъюнктных* пути (то есть пути, общими могут быть только начальная и конечная вершины). Таким образом, (a) и (b) эквивалентны.

(b) $\implies$ (c).
Пусть для любых двух вершин u и v существуют два пути P_1 и P_2, не имеющие общих вершин, кроме u и v. Тогда рассмотрим граф, полученный как объединение путей $P_1 \cup P_2$. Очевидно, что в этом объединении содержится цикл, поскольку P_1 и P_2 различны (не совпадают по внутренним вершинам). Следовательно, u и v принадлежат одному циклу, что и утверждается в условии (c).

(c) $\Longrightarrow$ (h).

Пусть любые две вершины лежат в некотором цикле. Рассмотрим произвольные три вершины u, v и w. Тогда по (c) найдётся цикл C, содержащий вершины u и v. Если $w \notin C$, то уже сам цикл является u–v путём, не проходящим через w. Если же $w \in C$, то на цикле C существуют два пути между u и v. Так как цикл проходит через w, один из этих двух путей не содержит вершины w (так как, проходя циклом, можно пойти «в другую сторону»). Таким образом, в любом случае найдётся путь из u в v, который не проходит через w, что и соответствует условию (h).

(h) $\Longrightarrow$ (a).

Предположим противное, то есть что в G существует точка сочленения v. Тогда граф $G \setminus \{v\}$ распадается хотя бы на две компоненты. Выберем две вершины u и w, принадлежащие разным компонентам $G \setminus \{v\}$. Тогда любой путь из u в w в графе G обязательно должен проходить через v. Однако условие (h) утверждает, что для любых трёх вершин, в частности для u, w и v, существует путь из u в w, не проходящий через v. Противоречие. Следовательно, если (h) выполняется, то точек сочленения в G быть не может, то есть условие (a) верно.

Вывод. Мы доказали следующие импликации:

$$(a) \iff (b) \implies (c) \implies (h) \implies (a).$$

Таким образом, условия (a), (b), (c) и (h) эквивалентны.

Задача N3

Условие задачи

Пусть дан связный неориентированный граф $G = (V, E)$. Требуется раскрасить все его рёбра таким образом, чтобы в каждом вершинно-простом цикле (то есть цикле, в котором внутренние вершины не повторяются) цвета всех рёбер были попарно различны. Необходимо найти минимальное число цветов, достаточное для такой раскраски, при этом алгоритм должен работать за $O(n + m)$ времени.

Замечание о структуре раскраски

Заметим, что требование, чтобы в любом простом цикле все рёбра имели различные цвета, эквивалентно тому, что *никакой цикл не может целиком лежать в одном классе цвета.* Иными словами, если мы рассмотрим множество рёбер одного цвета, то оно не должно содержать циклов, то есть должно быть аксиально *ацикличным* (в терминах неориентированных графов — лесом).

Таким образом, искомая раскраска — это разбиение множества рёбер E графа G на некоторое число подмножеств, каждое из которых является лесом.

Арборность графа

Минимальное число лесов, на которые можно разбить E, называется *арборностью* графа G и обозначается $\alpha(G)$. По теореме Нэша-Уильямса:

$$\alpha(G) = \max_{U \subseteq V,\, |U| \geq 2} \left\lceil \frac{|E(U)|}{|U| - 1} \right\rceil,$$

где $E(U)$ — множество рёбер, инцидентных вершинам U.

Необходимость и достаточность числа цветов

Необходимость. Пусть дана раскраска рёбер в C цветов, удовлетворяющая условию задачи. Тогда для каждого цвета i множество рёбер, раскрашенных в этот цвет, не содержит циклов (иначе в цикле были бы два ребра одинакового цвета). Иными словами, каждое такое множество является лесом. Таким образом, E представимо в виде объединения C лесов, что означает $C \geq \alpha(G)$.

Достаточность. Напротив, если граф G можно разбить на $\alpha(G)$ лесов, то, раскрасив рёбра каждого леса уникальным цветом, мы получим раскраску, в которой в каждом цикле рёбра принадлежат различным лесам, а значит, имеют различные цвета.

Таким образом, минимальное необходимое число цветов для искомой раскраски равно $\alpha(G)$.

Алгоритм раскраски за $O(n + m)$

Существует алгоритм, позволяющий разбить граф на $\alpha(G)$ лесов за $O(n + m)$ времени. Один из подходов основан на поиске *ориентации графа с ограниченной максимальной исходящей степенью* (например, методом последовательного удаления вершин минимальной степени). Такая ориентация даёт оценку $\alpha(G)$ сверху, а затем по алгоритмам разбиения можно построить требуемое разбиение на леса.

Итак, алгоритм можно описать следующими шагами:

1. Вычислить некоторую оценку арборности $\alpha(G)$ (например, с помощью жадного алгоритма удаления вершин, что работает за $O(n + m)$).
2. Разбить E на $\alpha(G)$ подмножеств так, чтобы каждое подмножество было аксиально ацикличным (то есть представляло собой лес).
3. Раскрасить все рёбра, присваивая каждой группе уникальный цвет.

В итоге, получаем раскраску рёбер с $\alpha(G)$ цветами, удовлетворяющую условию: в любом вершинно-простом цикле рёбра имеют попарно различные цвета.

Вывод

Минимальное число цветов, необходимое для раскраски рёбер связного графа G таким образом, чтобы в любом вершинно-простом цикле все рёбра имели различные цвета, равно арборности графа $\alpha(G)$, то есть

$$\text{минимальное число цветов} = \alpha(G) = \max_{U \subseteq V,\, |U| \geq 2} \left\lceil \frac{|E(U)|}{|U| - 1} \right\rceil.$$

Алгоритм построения такой раскраски работает за $O(n + m)$.

Задача N2

Условие задачи

Пусть дан связный неориентированный граф $G = (V, E)$. Необходимо назначить направления для как можно большего числа рёбер так, чтобы в получившемся ориентированном (или смешанном) графе выполнялось условие сильной связности, то есть из каждой вершины существует путь (с учетом ориентации) в каждую другую вершину. Алгоритм должен работать за $O(n + m)$.

Основная идея

По теореме Роббинса известно, что связный неориентированный граф допускает сильносвязную ориентацию, если и только если он 2-реберносвязен (то есть не содержит мостов). Поэтому:

- В **2-реберносвязных частях графа** (без мостов) можно назначить ориентацию для всех рёбер произвольным образом, и получившаяся ориентация будет сильносвязной.
- Ребро, являющееся мостом, при его ориентации нарушит сильную связность, так как оно единственным образом соединяет две части графа. Поэтому такие рёбра следует оставить неориентированными (или рассматривать как двунаправленные).

Таким образом, максимальное число рёбер, которое можно ориентировать, равно

$$|E| - b,$$

где b — число мостов в графе.

Алгоритм

1. **Нахождение мостов.** Используем стандартный алгоритм на основе поиска в глубину (DFS) для поиска мостов. Он работает за $O(n + m)$ и позволяет пометить все ребра, являющиеся мостами.

2. **Ориентирование рёбер.** Для каждого ребра $e \in E$:

   - Если e *не является мостом*, назначаем ему произвольную ориентацию (например, из вершины с меньшим номером в вершину с большим номером или согласно порядку обнаружения DFS).
   - Если e является мостом, оставляем его неориентированным (то есть допускаем прохождение в обоих направлениях).

   Таким образом, все не-мостовые рёбра будут ориентированы, а мосты — оставлены двунаправленными, что обеспечивает возможность прохождения в обоих направлениях между компонентами.

3. **Проверка сильной связности.** В 2-реберносвязных частях (без мостов) ориентированные рёбра обеспечивают сильную связность (по теореме Роббинса). Мосты, оставленные неориентированными, позволяют обеспечить связь между такими частями в обе стороны. Поэтому в получившемся графе из каждой вершины существует путь в каждую другую.

Асимптотика

- Поиск мостов работает за $O(n + m)$.
- Обход всех рёбер для назначения ориентации занимает $O(m)$.

Общая асимптотика алгоритма: $O(n + m)$.

Вывод

Максимально можно ориентировать все рёбра, за исключением мостов, то есть ровно

$$|E| - (\text{число мостов})$$

ребер. При этом оставив мосты неориентированными, мы сохраняем условие сильной связности, то есть из каждой вершины существует путь до каждой другой. Алгоритм работает за $O(n + m)$.

Задача N5

Условие задачи

У Коли есть n коробочек и nk разноцветных камней, причем для каждого из n цветов имеется ровно k камней этого цвета. Коробочки окрашены в те же n цветов. Изначально в каждой коробочке лежит ровно k камней (смешанных по цветам). Необходимо переставить камни так, чтобы в каждой коробке оказались *ровно* камни соответствующего цвета. За одно действие разрешается:

- перенести один камень из одной коробки в другую, или
- переместить пустую руку к другой коробке.

При этом движение руки к первой используемой коробке не считается действием.

Требуется найти минимальное число действий, при этом алгоритм должен работать за $O(nk)$.

Идея решения

Обозначим коробки и камни следующим образом:

- Пронумеруем коробки от 1 до n; коробка i имеет цвет i.
- Пусть a_{ij} — число камней цвета j, находящихся в коробке i.

В *правильном состоянии* для каждой коробки i должно быть:

$$a_{ii} = k \quad \text{и} \quad a_{ij} = 0 \text{ для } j \neq i.$$

Следовательно, в коробке i *лишними* являются камни, цвета отличного от i; их число равно:

$$d_i = k - a_{ii}.$$

Общее число камней, которые нужно переместить, составляет

$$M = \sum_{i=1}^{n} d_i = nk - \sum_{i=1}^{n} a_{ii}.$$

Каждый такой камень следует переместить из коробки, где он лишний, в коробку, чей цвет он представляет.

Цепочки обменов.

Заметим, что многие перемещения можно организовать в *циклические обмены*. Например, если:

- в коробке i лишний камень цвета j (то есть $a_{ij} > 0$),
- а в коробке j лишний камень цвета ℓ,

то можно последовательно переместить камень из i в j, затем камень из j в ℓ и т. д., пока в какой-то момент не вернемся в коробку i. При такой организации обмена:

- Каждый перенос камня (с его перемещением) выполняется за одно действие.
- Если же обмен образует цикл, то можно *цепочкой* выполнить все переносы, так что рука, уже держа камень, сразу перемещается к следующей коробке без необходимости отдельно передвигать пустую руку.

Однако, если обмены образуют несколько *независимых циклов*, то для перехода от завершения одного цикла к началу другого потребуется дополнительное действие для перемещения пустой руки (перемещение между циклами). При этом для первого цикла перемещение руки не требуется (начальное положение руки выбирается бесплатно).

Пусть оптимальная схема обменов разбивает все необходимые переносы на c циклов. Тогда:

- Требуется ровно $M = nk - \sum_{i=1}^{n} a_{ii}$ действий на перенос камней.
- Дополнительно требуется $c-1$ действий на перемещения пустой руки между циклами.

Таким образом, минимальное число действий:

$$\boxed{(nk - \sum_{i=1}^{n} a_{ii}) + (c-1).}$$

Алгоритмическая реализация за $O(nk)$

Можно действовать следующим образом:

1. Для каждой коробки i просуммировать камни правильного цвета, получив a_{ii} и вычислить $d_i = k - a_{ii}$. Это требует обхода всех nk камней.
2. Для каждой коробки i и для каждого $j \neq i$ фиксировать количество камней a_{ij} — камней цвета j в коробке i. Эти камни необходимо переместить в коробку j.
3. Построить граф или список *требуемых перемещений* между коробками. Каждый перенос (из i в j) будет представлен a_{ij} раз.
4. Разбить набор перемещений на циклические обмены. Это можно сделать с помощью обхода по «сети обменов». Общее число элементов в этой сети равно $M = nk - \sum a_{ii}$, а обход производится за $O(nk)$.
5. Пусть найдено c циклов. Тогда ответ равен $M + (c-1)$.

Вывод

Минимальное число действий, которое необходимо выполнить, равно

$$\boxed{(nk - \sum_{i=1}^{n} a_{ii}) + (c-1),}$$

где c — число циклов в оптимальном разбиении перемещений камней между коробками. Такой алгоритм работает за $O(nk)$.

Задача N6

Формулировка задачи

Граф $G = (V, E)$ называется *расщепляемым*, если его множество вершин V можно разбить на две части C и I ($V = C \dot\cup I$) так, что:

- C — клика (каждая пара вершин смежна),
- I — независимое множество (никакие две вершины не соединены ребром).

Задача состоит в том, чтобы за время $O(n + m)$ проверить, является ли граф G расщепляемым, и если да, то найти одно из таких разбиений.

Идея решения

Известна характеристика расщепляемых графов: G является расщепляемым, если его вершины можно упорядочить по невозрастанию степеней так, что для некоторого порога t первые t вершин образуют клику, а оставшиеся $n - t$ вершин — независимое множество.

Алгоритм состоит из следующих этапов:

1. **Сортировка вершин по степеням.** Поскольку степени не превосходят n, можно выполнить *подсчёт сортировку* за $O(n + m)$ (при условии, что граф задан списками смежности).

2. **Выбор кандидата на клику.** Пусть $v_1, v_2, \ldots, v_n$ — вершины, отсортированные по невозрастанию степени. Предположим, что существует такое t, что:

   $$C = \{v_1, v_2, \ldots, v_t\} \quad \text{и} \quad I = \{v_{t+1}, \ldots, v_n\}.$$

   При этом необходимо, чтобы:

   (a) Подграф, индуцированный на C, был полным, то есть для любой пары $v_i, v_j \in C$ ребро $\{v_i, v_j\}$ принадлежит E.

   (b) Подграф, индуцированный на I, был пустым, то есть для любых $u, v \in I$ ребра $\{u, v\}$ нет.

   Обычно выбирают t как наибольшее такое число, что *каждая вершина v_i при $1 \le i \le t$ имеет внутри C не менее $t - 1$ соседей.* (Заметим, что если граф расщепляем, то существует такое разбиение.)

3. **Проверка кандидата.** Для кандидата C и $I = V \setminus C$ проверяем:

   - **Клика:** Для каждой вершины $v \in C$ убедиться, что все остальные вершины $C \setminus \{v\}$ являются её соседями.
   - **Независимость:** Для каждой вершины $u \in I$ проверить, что среди её соседей нет другой вершины из I.

   Если обе проверки проходят, граф расщепляем, и найденное разбиение является искомым.

Алгоритм в псевдокоде

Algorithm 1 Проверка расщепляемости графа и нахождение разбиения

Require: Граф $G = (V, E)$ с n вершинами и m ребрами
Ensure: Разбиение $V = C \dot\cup I$, если граф расщепляем, иначе отказ

Вычислить степени всех вершин, $d(v)$ для $v \in V$
Отсортировать вершины по невозрастанию степеней, получив упорядочение $v_1, v_2, \ldots, v_n$
Найти наибольшее t такое, что для каждого $1 \le i \le t$ выполнено:

$$\text{число соседей } v_i \text{ в } \{v_1, \ldots, v_t\} \ge t - 1.$$

Положить $C \leftarrow \{v_1, \ldots, v_t\}$ и $I \leftarrow V \setminus C$
for all $v \in C$ **do**
    for all $u \in C$, $u \neq v$ **do**
        if $\{u, v\} \notin E$ **then**
            return «Граф не является расщепляемым»
        end if
    end for
end for
for all $u \in I$ **do**
    for all $w \in I$, $w \neq u$ **do**
        if $\{u, w\} \in E$ **then**
            return «Граф не является расщепляемым»
        end if
    end for
end for
return Разбиение C и I

Анализ сложности

- Вычисление степеней всех вершин за $O(n + m)$.
- Подсчёт сортировки по степеням выполняется за $O(n + m)$ с использованием сортировки подсчётом (так как степени не превосходят n).
- Выбор порога t производится за $O(n)$.
- Проверка, что C является кликой и I — независимым множеством, сводится к обходу списков смежности, что в сумме занимает $O(n+m)$.

Таким образом, общая асимптотика алгоритма равна $O(n + m)$.

Вывод

Если граф G является расщепляемым, то описанный алгоритм за $O(n + m)$ времени находит разбиение $V = C \dot\cup I$ такое, что C — клика, а I —

независимое множество. Если же граф не удовлетворяет условию, алгоритм сообщает, что G не является расщепляемым.